

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L3: Fundamentals of assembler programming (I) **Computer Structure**

Bachelor in Computer Science and Engineering
Bachelor in Applied Mathematics and Computing
Dual Bachelor in Computer Science and Engineering and Business Administration

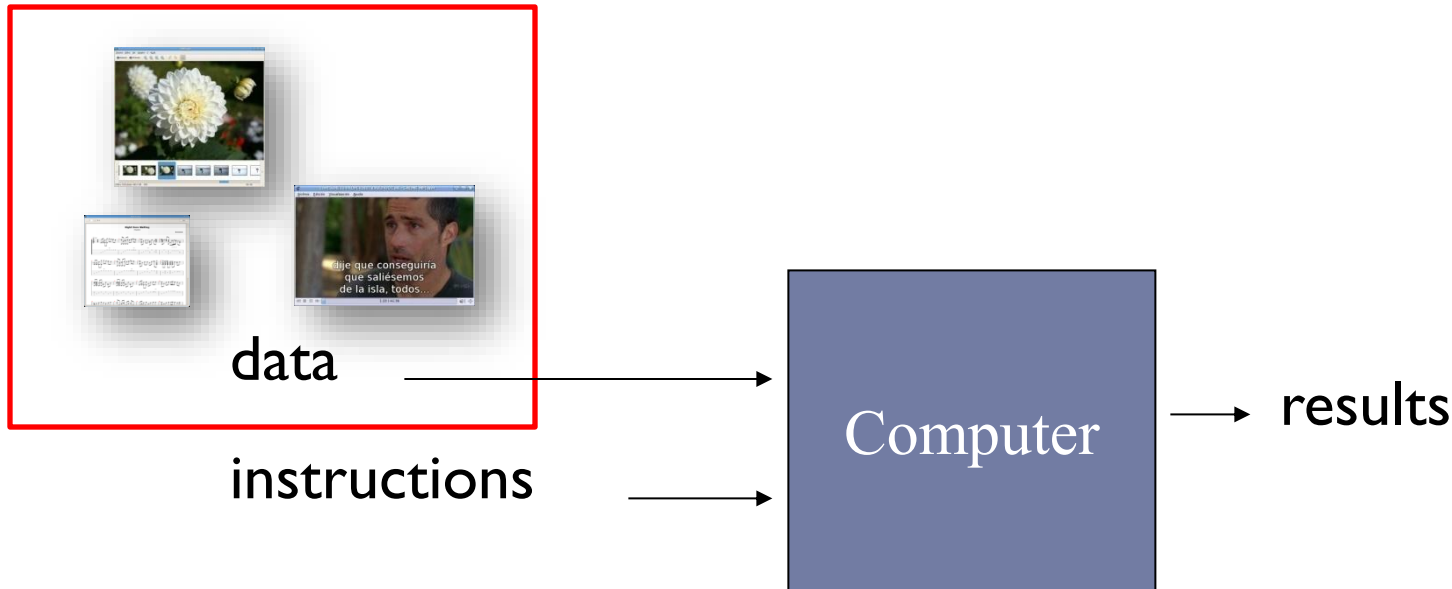


Contents

1. Basic concepts on assembly programming
 1. Motivations and goals
 2. Introduction to RISC-V32
2. RISC-V32 assembly language, memory model and data representation
3. Instruction formats and addressing modes
4. Floating point in assembly
5. Procedure calls and stack convention

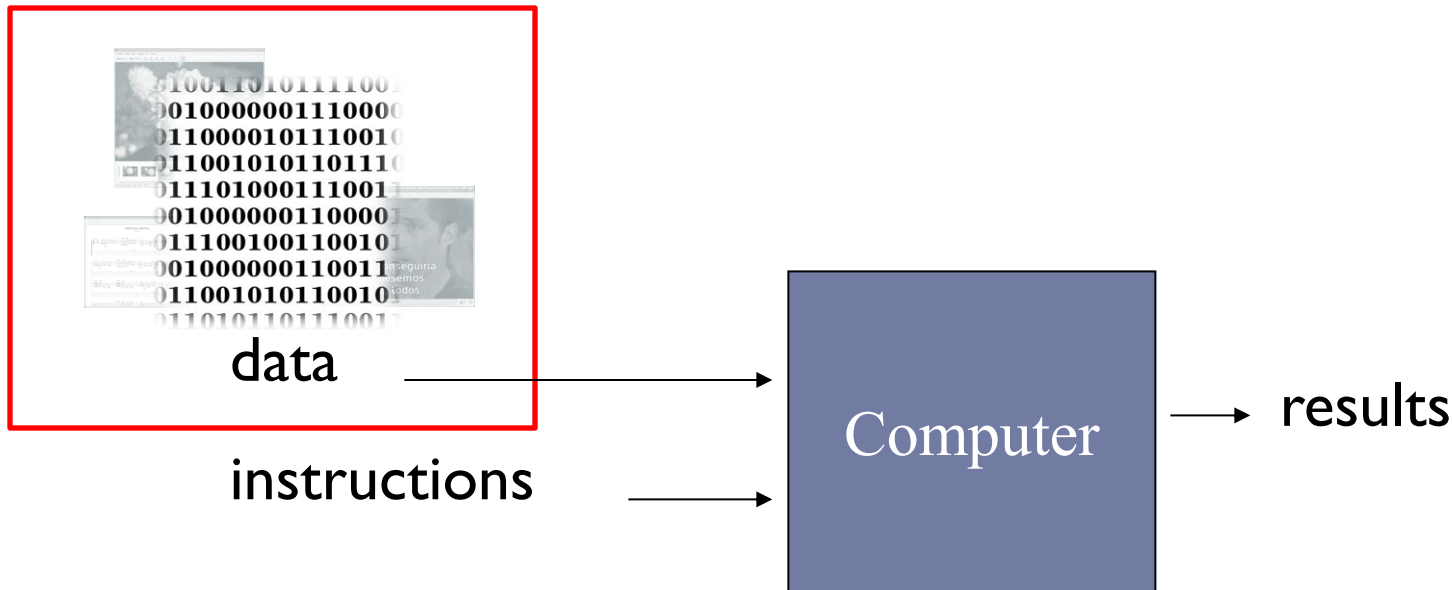
Types of information: instructions and data

► Data representation...



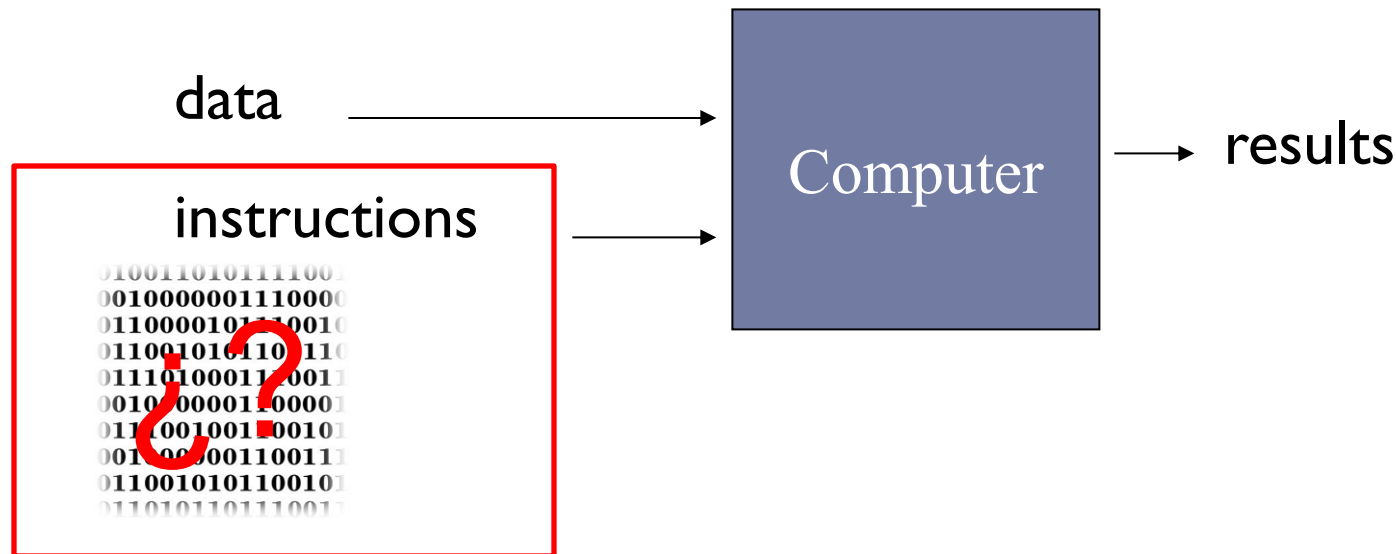
Types of information: instructions and data

► Binary data representation.



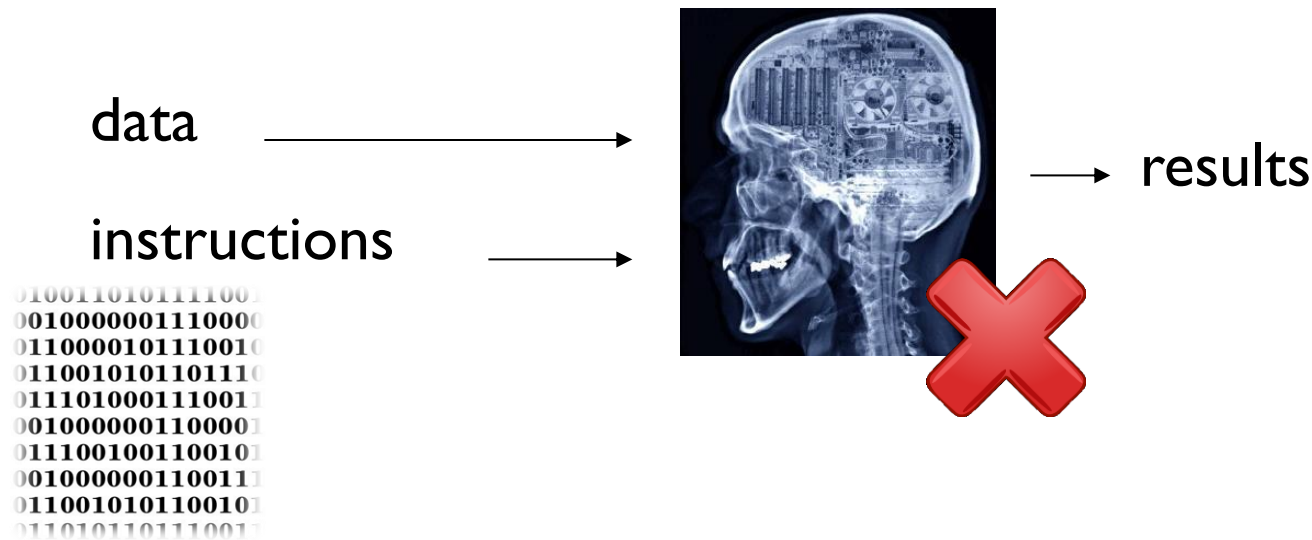
Types of information: instructions and data

- ▶ What about the instructions?
 - ▶ Machine instructions, properties and format



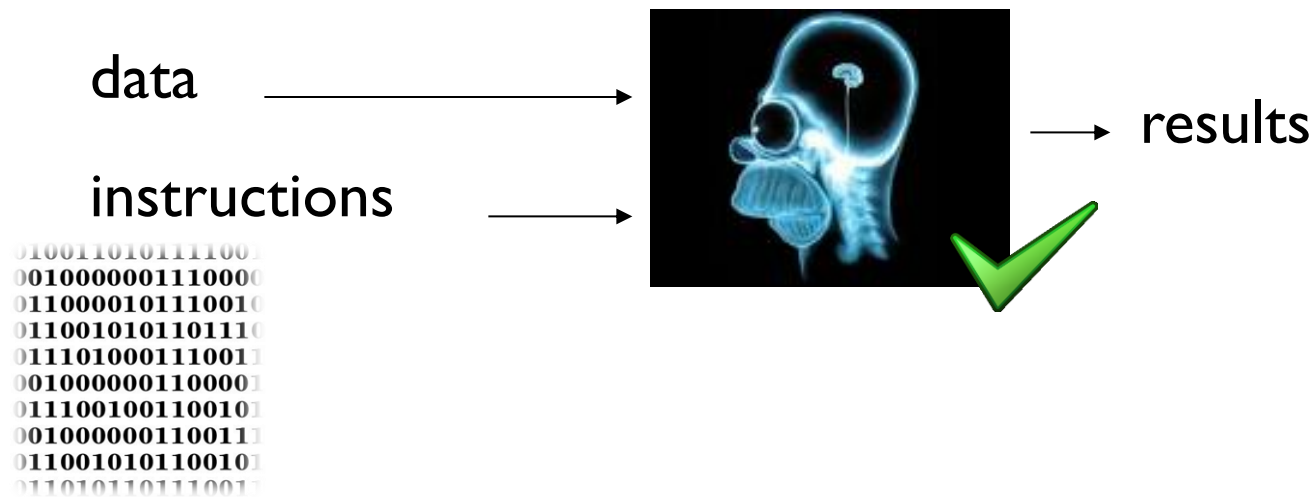
Machine instructions

- ▶ There are not complex instructions...



Machine instructions

- ▶ ... but very simple tasks...



Machine instruction: definition

- ▶ **Machine instruction:** elementary operation that can be executed directly by the processor.

- ▶ Ej.: $Rd = Rs + Imm$ # `addi Rd, Rs, Imm`

0010011 dddddd 000 sssss iiiii iiiii iiiii

Machine instruction: definition

- ▶ **Machine instruction:** elementary operation that can be executed directly by the processor.

- ▶ Ej.: $Rd = Rs + Imm$ # `addi Rd, Rs, Imm`

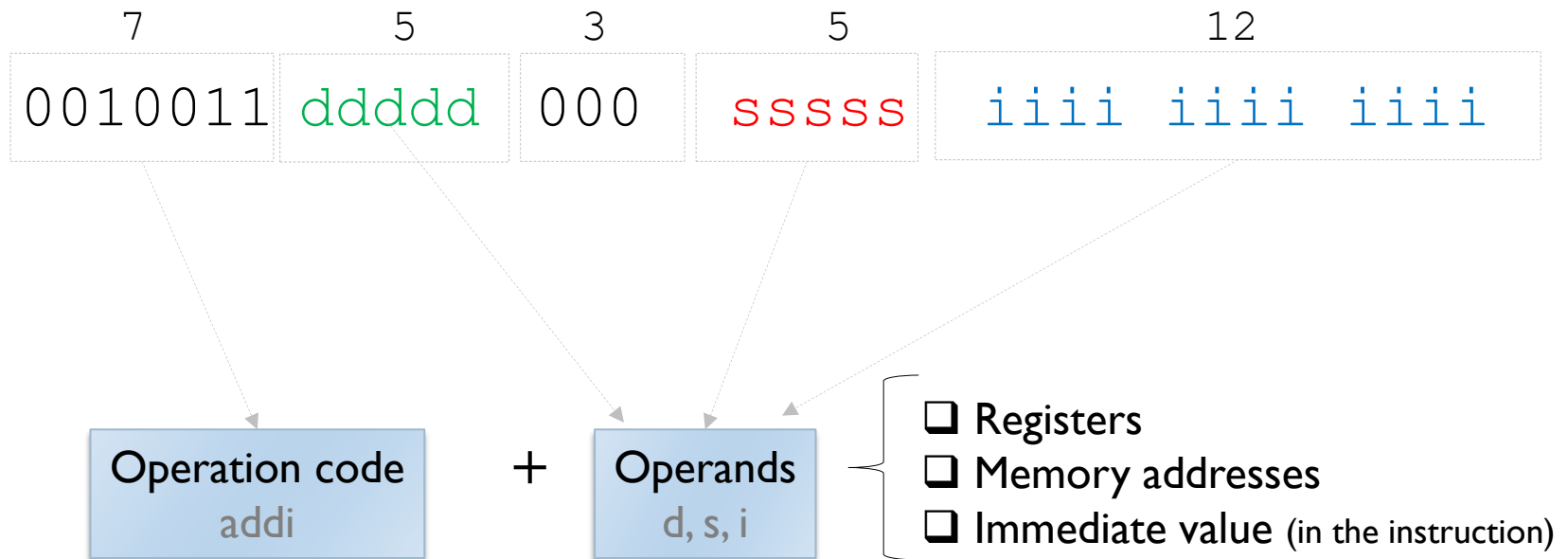
0010011 dddddd 000 sssss iiiii iiiii iiiii

RISC-V instruction encoder:

- ▶ <https://luplab.gitlab.io/rvcodecs/>

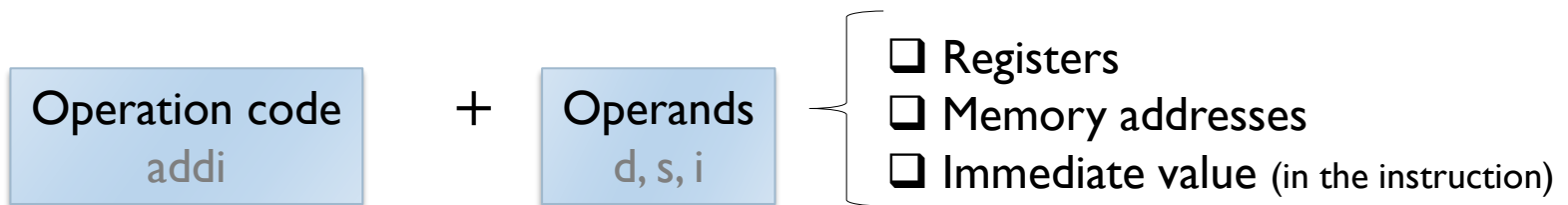
Machine instruction: properties

- ▶ Perform a **single, simple task**
- ▶ Operate on a **fixed number of operands**
- ▶ **Include all** the **information necessary** for its **execution**



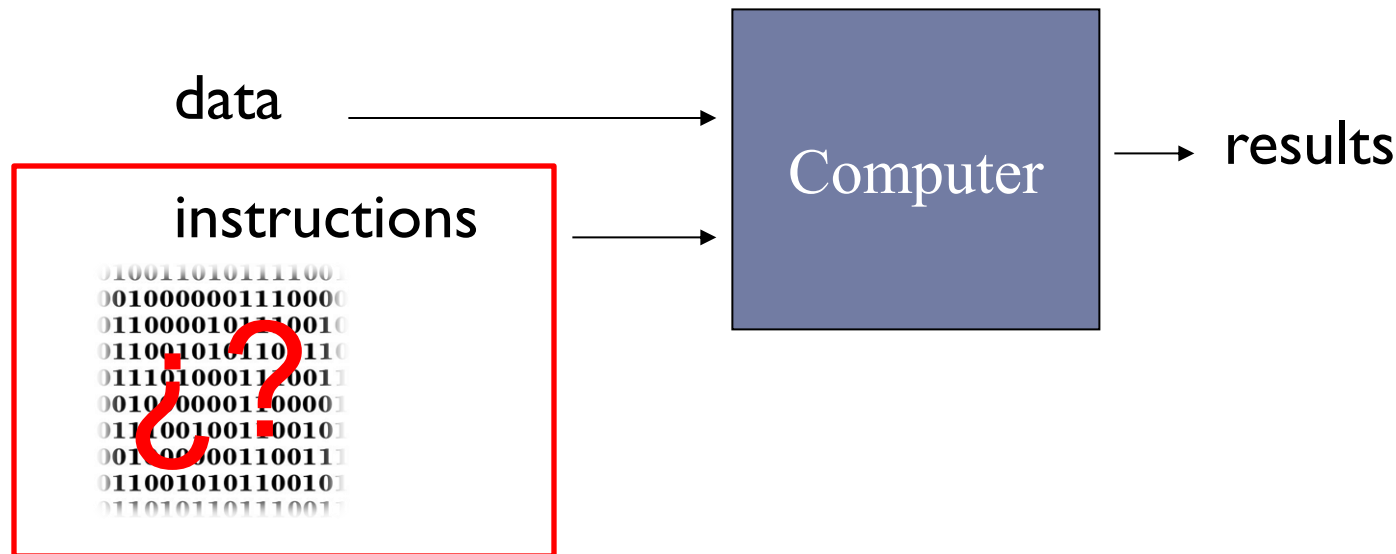
Machine instruction: included information

- ▶ The **operation to be performed**.
- ▶ Where the **operands** are located:
 - ▶ In registers
 - ▶ In memory
 - ▶ In the instruction itself (immediate)
- ▶ Where to leave the **results** (as operand)
- ▶ A reference to the **next instruction** to be executed
 - ▶ Implicitly: the following instruction
 - ▶ A program is a consecutive sequence of machine instructions.
 - ▶ Explicitly in branching instructions (as operand)



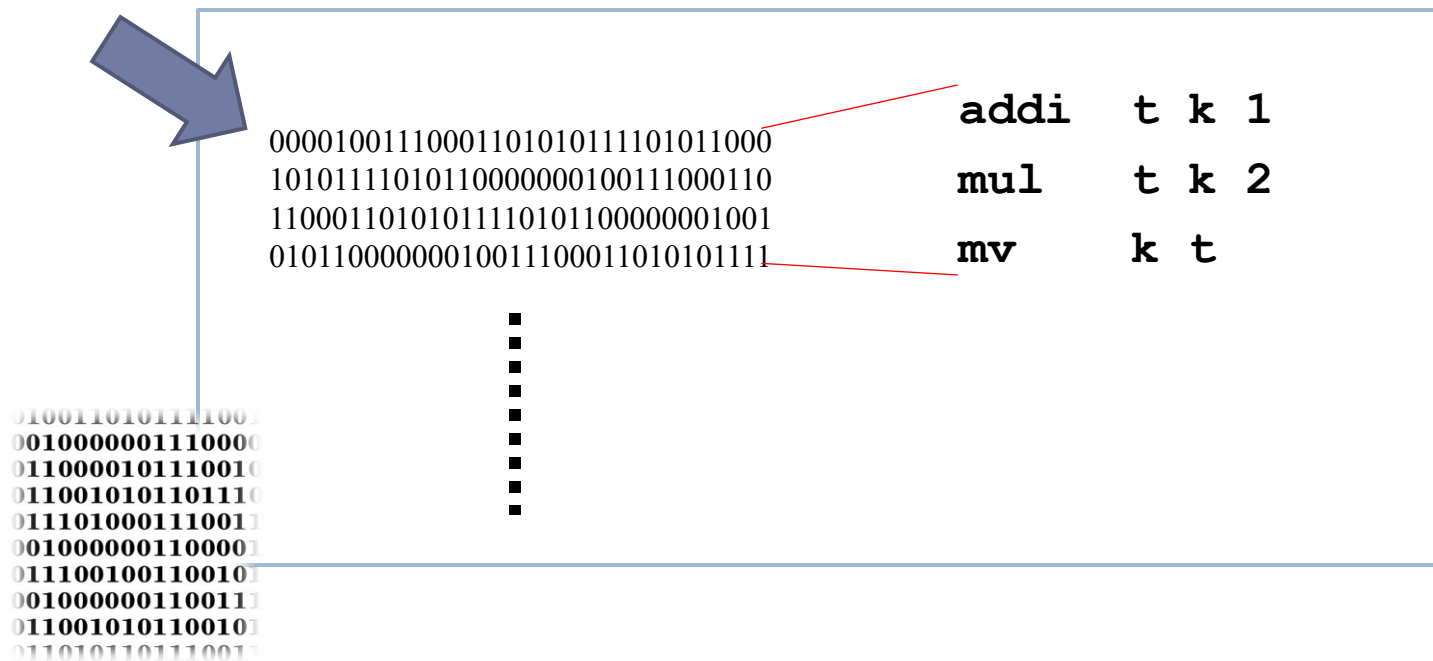
Types of information: instructions and data

- ▶ What about the instructions?
 - ▶ Program, assembly language, ISA



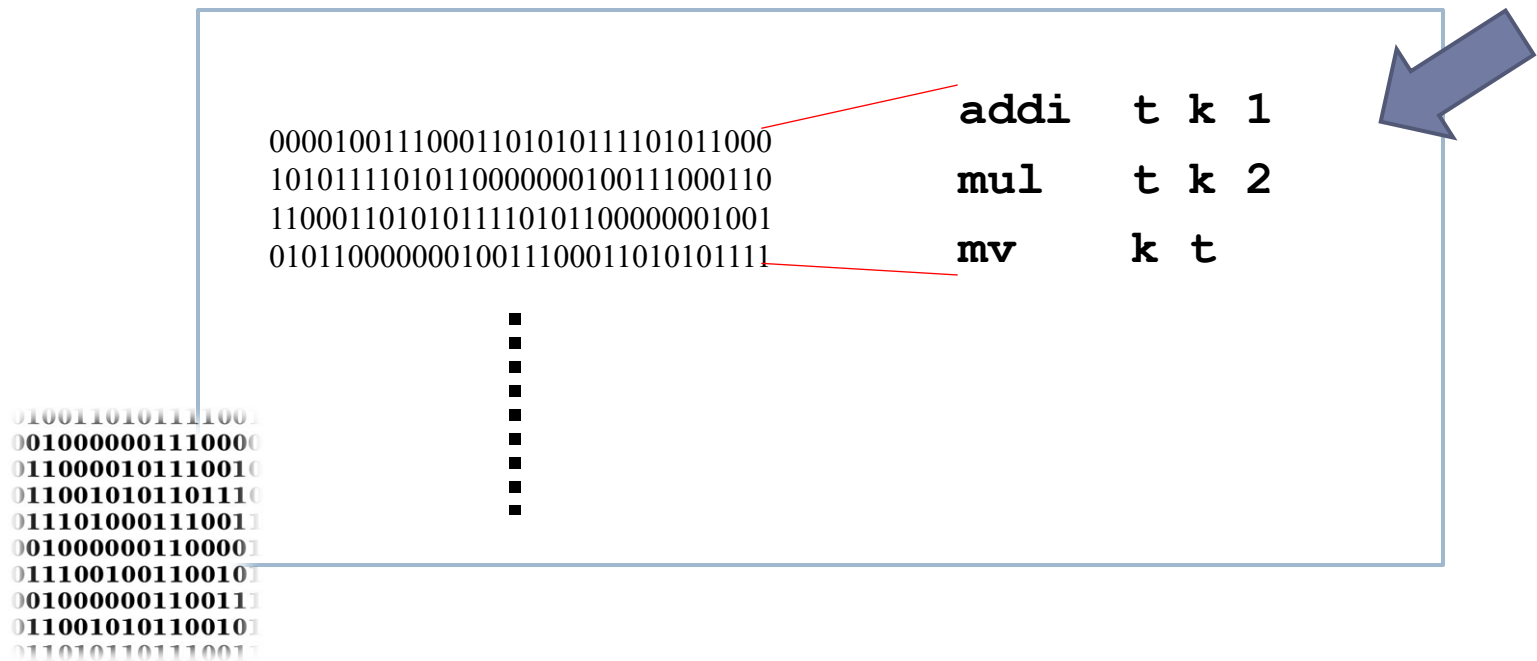
Definition of program

- **Program:** Ordered sequence of machine instructions that are executed by default in order.



Assembly language definition

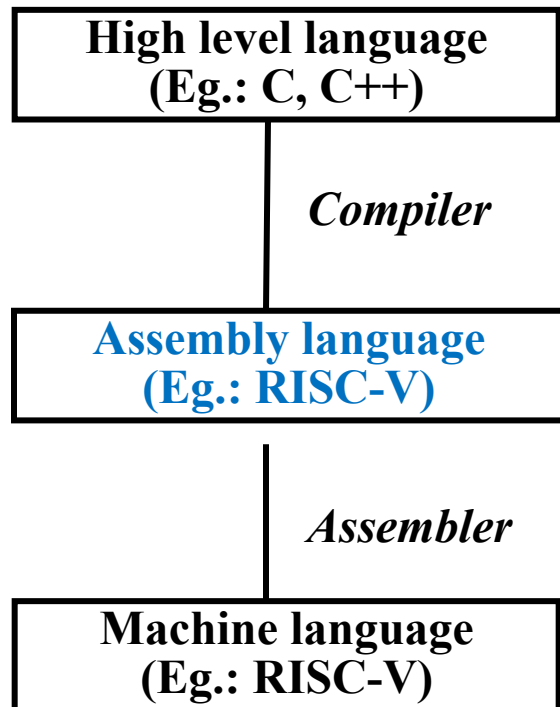
- **Assembly language:** programmer-readable language that is **the most direct representation of architecture-specific machine code.**



Assembly language definition

- ▶ **Assembly language:** programmer-readable language that is **the most direct representation of architecture-specific machine code.**
- ▶ Uses symbolic codes to represent instructions:
 - ▶ `add` – addition
 - ▶ `lw` – Load a memory data
- ▶ Uses symbolic codes for data and references:
 - ▶ `t0` – register identifier
 - ▶ `(t0)` – memory placement identifier
- ▶ There is an assembly instruction per machine instruction:
 - ▶ `add t1, t2, t3`

Languages levels



```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

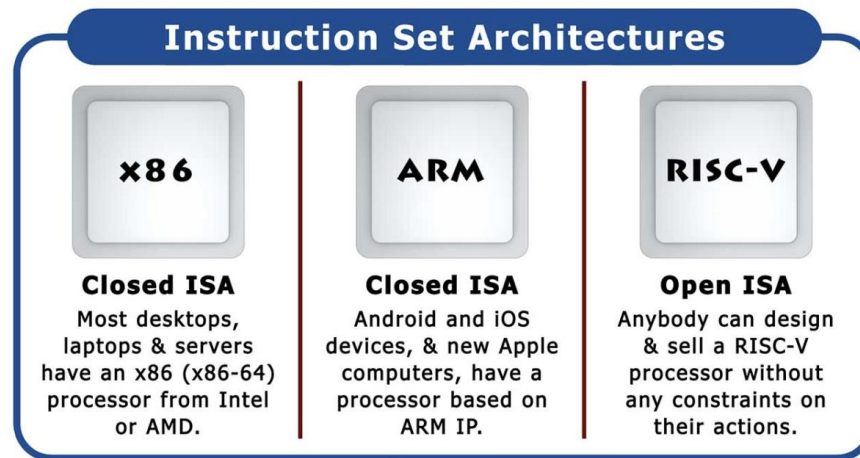
```
lw    t0, 0(x2)  
lw    t1, 4(x2)  
sw    t1, 0(x2)  
sw    t0, 4(x2)
```

```
0000 1001 1100 0110 1010 1111 0101 1000  
1010 1111 0101 1000 0000 1001 1100 0110  
1100 0110 1010 1111 0101 1000 0000 1001  
0101 1000 0000 1001 1100 0110 1010 1111
```

Instruction sets

- ▶ **Instruction Set Architecture (ISA)**
 - ▶ Instruction set of a processor
 - ▶ Boundary between hardware and software
- ▶ **Examples:**
 - ▶ x86, x86-64, ARM, MIPS, RISC-V, PowerPC, Etc.

- ▶ **Types:**



Characteristics of an instruction set (1 / 2)

- ▶ **Format and coding of the instruction set:**
 - ▶ Fixed or variable length instructions
 - ▶ 80x86: variable (from 1 up to 18 bytes)
 - ▶ MIPS, ARM: fixed
- ▶ **Operands:**
 - ▶ Registers, memory, the instruction itself
- ▶ **Type and size of operands:**
 - ▶ bytes: 8 bits
 - ▶ integers: 16, 32, 64 bits
 - ▶ floating-point numbers: single precision, double precision, etc.
- ▶ **Addressing modes:**
 - ▶ They specify where and how to access operands (register, memory or the instruction itself)

Characteristics of an instruction set (2/2)

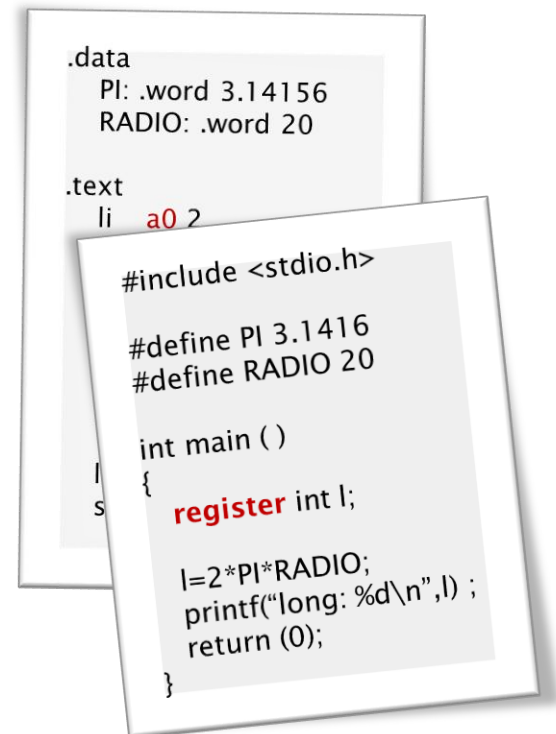
- ▶ **Operations:**
 - ▶ Arithmetic, logic, transfer, control, control, etc.
- ▶ **Flow control instructions:**
 - ▶ Unconditional jumps
 - ▶ Conditional jumps
 - ▶ Procedure calls
- ▶ **Memory addressing:**
 - ▶ Most of them use byte addressing
 - ▶ They provide instructions for accessing multi-byte elements from a given position

Programming model of a computer

- ▶ A computer offers a programming model that consists of:
 - ▶ **Instruction set (assembly language)**
 - ▶ *ISA: Instruction Set Architecture*
 - ▶ An instruction includes:
 - Operation code
 - Other elements: registers, memory address, numbers
 - ▶ **Storing elements**
 - ▶ Registers
 - ▶ Memory
 - ▶ Registers of I/O controllers
 - ▶ **Execution modes**

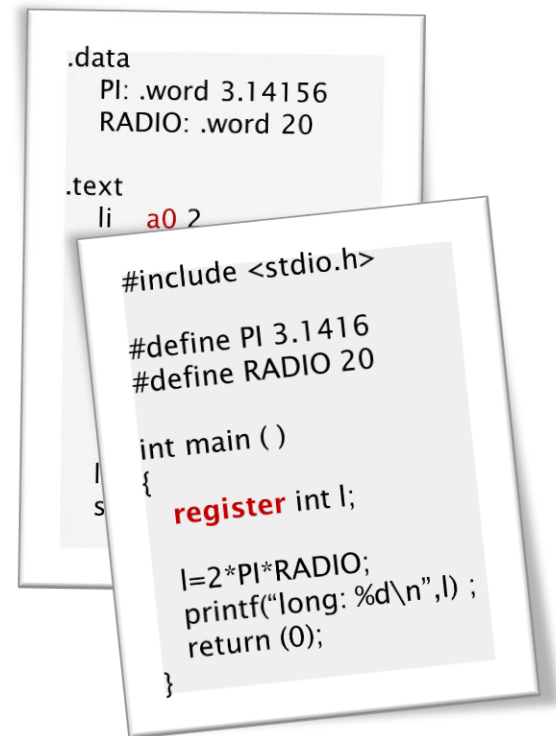
Goals

- ▶ To know how the elements of a high-level assembly language are represented.:
 - ▶ Data types (int, char, ...)
 - ▶ Control structures (if, while, ...)
- ▶ To be able to write small programs in assembler
- ▶ To work with open-hardware:
 - ▶ RISC-V
 - ▶ Simulated by CREATOR

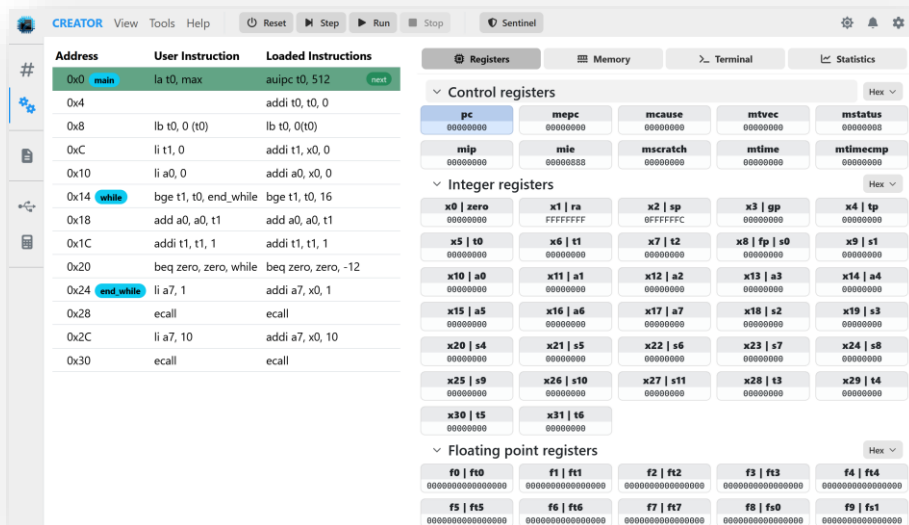


Motivation to learn assembly

- ▶ Understand how high-level languages are executed
 - ▶ C, C++, Python, Java, ...
- ▶ Analyze the execution time of high-level instructions.
- ▶ Useful in specific domains:
 - ▶ Compilers
 - ▶ Operating Systems
 - ▶ Games
 - ▶ Embedded systems
 - ▶ Etc.



Motivation to use CREATOR simulator

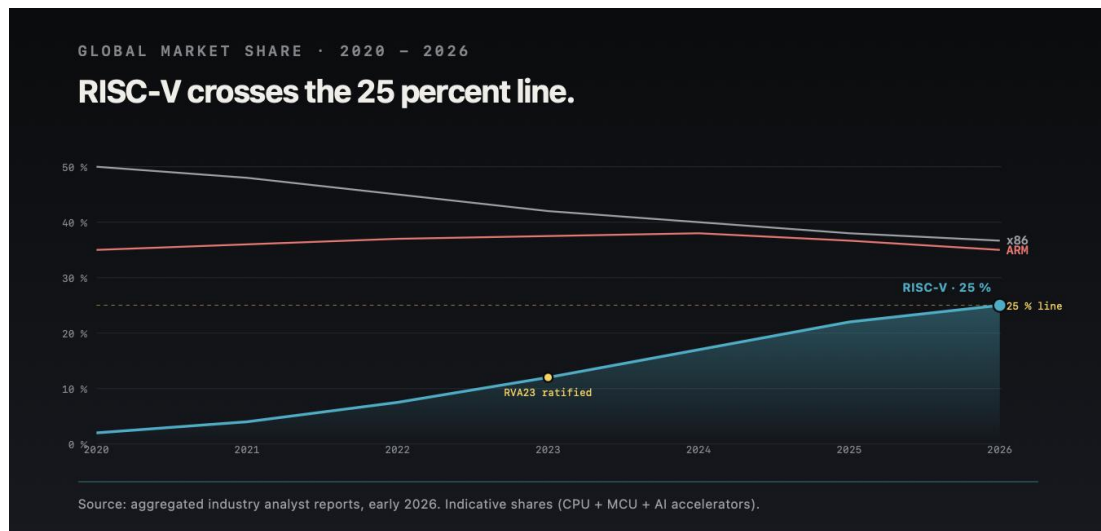


<https://creatorsim.github.io/>

- ▶ CREATOR: didaCtic geneRic assEmbly progrAmming simulaTOR
- ▶ CREATOR can simulate MIPS₃₂ and RISC-V architectures
- ▶ CREATOR can be executed from Firefox, Chrome, Edge or Safari

Advantages of using RISC-V (1/2)

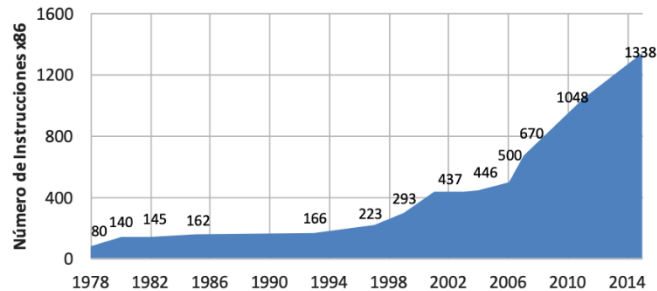
- ▶ Open hardware architecture:
 - ▶ Allows anyone to design, manufacture and sell RISC-V chips and software
- ▶ RISC processor (Reduced Instruction Set Computer)
 - ▶ RISC-V, ARM, MIPS, etc.
- ▶ Fast growing market share:



Motivations to use RISC-V (2/2)

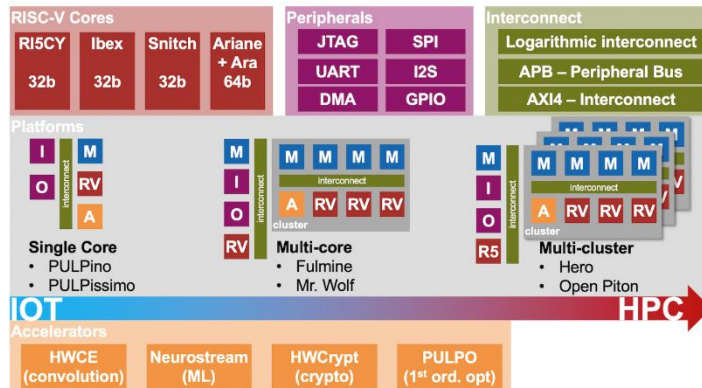
► Small and simple set of instructions

- RV32I -> ~47 instructions, RV32IMAF -> ~76
- Difference with x86 architecture instructions



Guía Práctica de RISC-V.
David Patterson y Andrew Waterman

► Modular instruction set:



https://pulp-platform.org/img/main/pulp_family.png

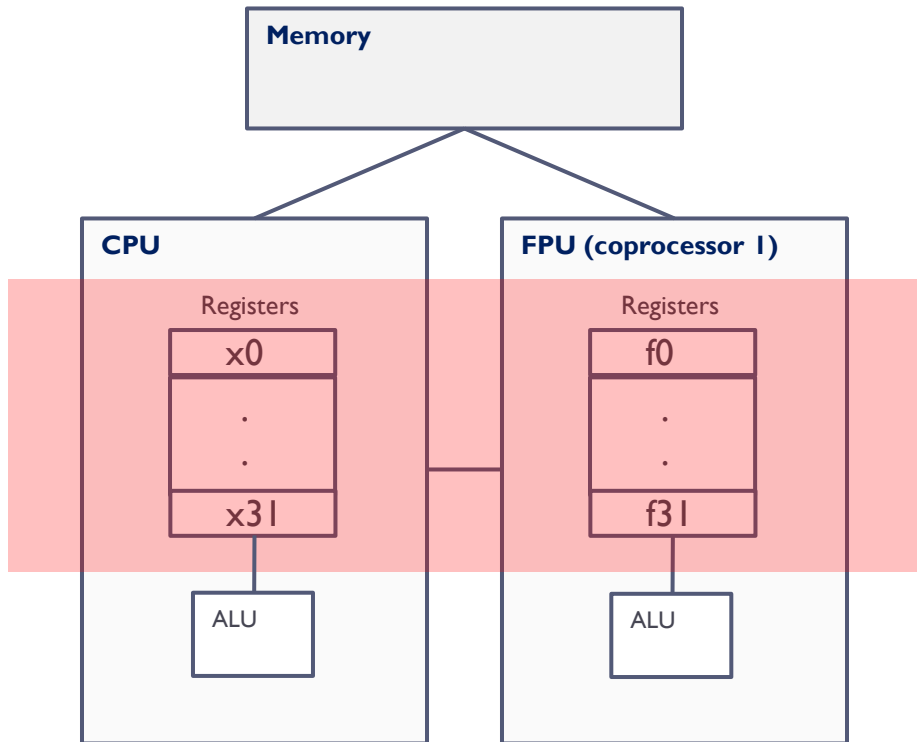
RISC-V instruction set

- ▶ Instruction sets:
 - ▶ RV32I: Set of instructions on integers. 32 bits
 - ▶ RV64I: Set of instructions on integers. 64 bits
 - ▶ RV128I: Set of instructions about integers. 128 bits
- ▶ On each of them there are different extensions:
 - ▶ M: instructions for integer multiplication and division
 - ▶ F: instructions for single-precision floating point
 - ▶ D: double precision floating-point instructions
 - ▶ G: Includes M, F and D
 - ▶ Q: quadruple-precision floating-point instructions
 - ▶ Etc.
- ▶ Example: RV64IF: RISC-V 64-bits processor with simple-precision floating-point instructions

Contents

1. Basic concepts on assembly programming
 1. Motivations and goals
 2. Introduction to RISC-V32
 - ▶ Registers
 - ▶ Data Transfer
 - ▶ Arithmetic and Logical Operations
 - ▶ Shifts
 - ▶ Branches or Jumps
2. RISC-V32 assembly language, memory model and data representation
3. Instruction formats and addressing modes
4. Floating point in assembly
5. Procedure calls and stack convention

RISC-V 32



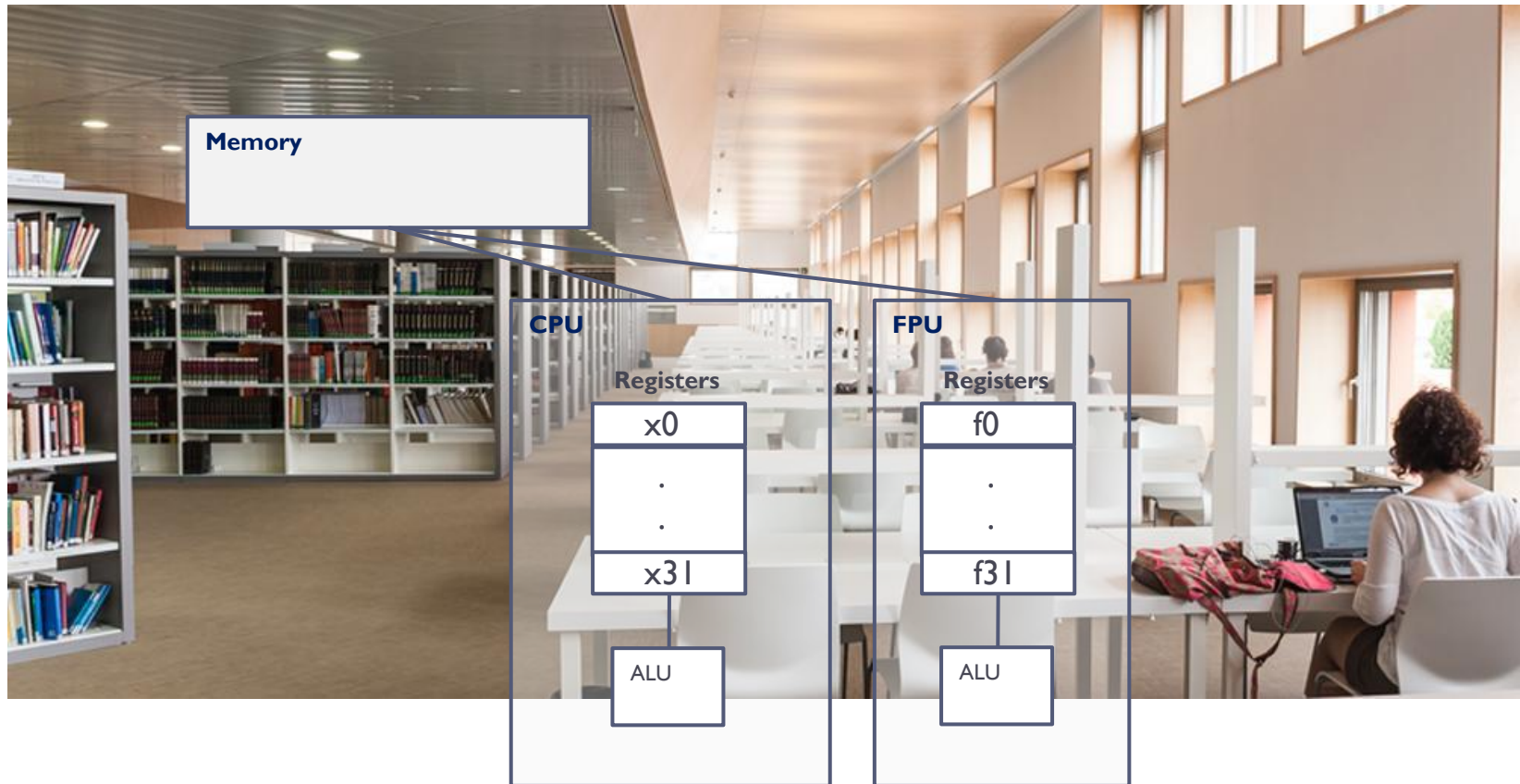
▶ Registers

- ▶ Inside the processor
- ▶ Instructions work with values in registers
- ▶ 32 registers of 32 bits

▶ Memory

- ▶ Outside the processor
- ▶ Data exchange to/from registers
- ▶ More capacity but longer access time than registers
- ▶ 32-bit addresses
 - ▶ 4 GiB addressable

RISC-V 32 (memory and registers)



RISC-V 32 register file

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

- 32 registers
 - 4 bytes of size (one word)
 - Name starts with **x** at the beginning
- Usage Convention
 - Reserved
 - Arguments
 - Results
 - Temporary
 - Pointers

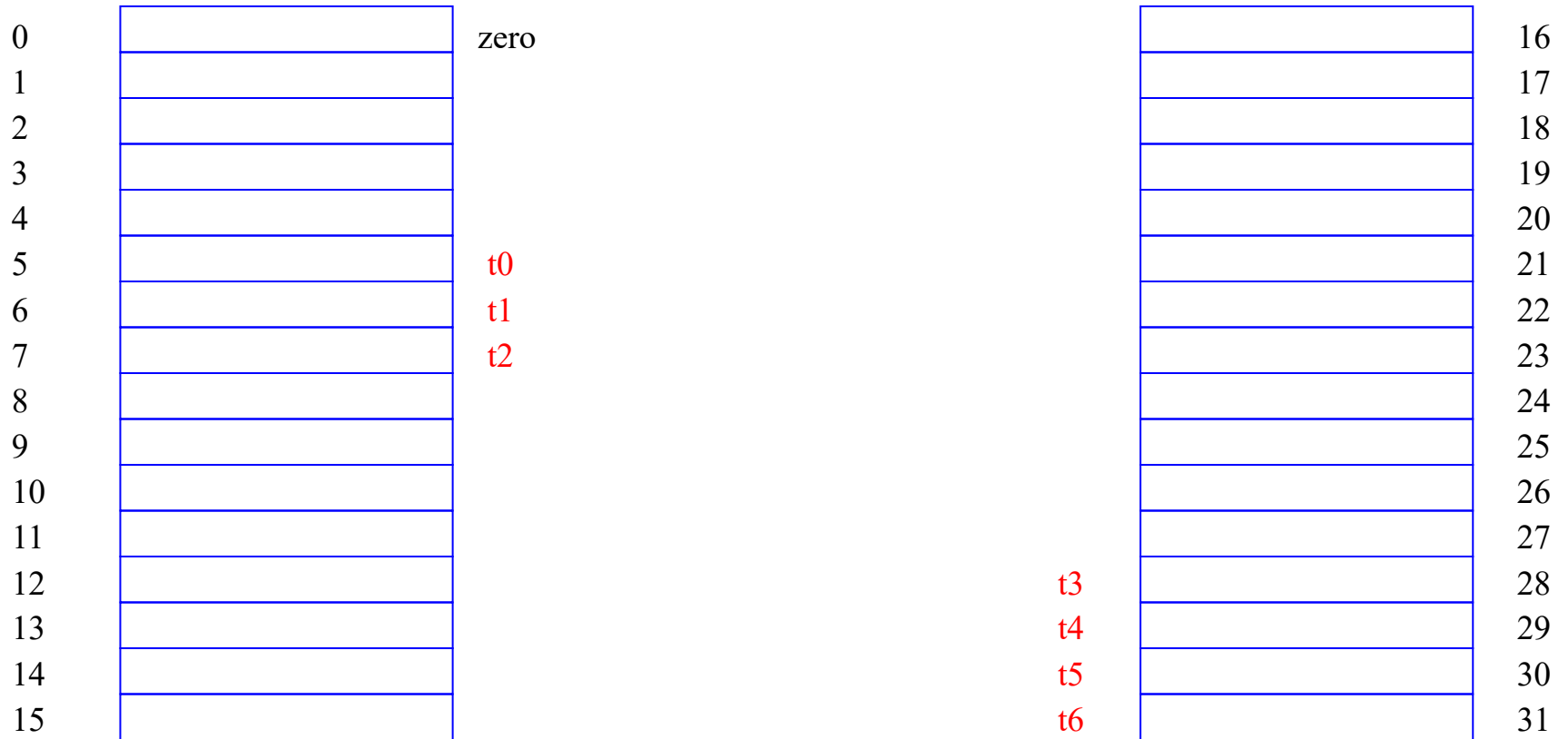
	16
	17
	18
	19
	20
	21
	22
	23
	24
	25
	26
	27
	28
	29
	30
	31

RISC-V 32 register file

0		zero		16
1				17
2				18
3				19
4				20
5				21
6				22
7				23
8				24
9				25
10				26
11				27
12				28
13				29
14				30
15				31

Hardwired to zero
Cannot be changed

RISC-V 32 register file



Temporary registers

RISC-V 32 register file

0		zero		16
1				17
2				18
3				19
4				20
5		t0	s2	21
6		t1	s3	22
7		t2	s4	23
8		s0 / fp	s5	24
9		s1	s6	25
10			s7	26
11			s8	27
12			s9	28
13			s10	29
14			s11	30
15			t3	31
			t4	
			t5	
			t6	

Preserved values

RISC-V 32 register file

0		zero			
1		ra	a6		16
2			a7		17
3			s2		18
4			s3		19
5		t0	s4		20
6		t1	s5		21
7		t2	s6		22
8		s0 / fp	s7		23
9		s1	s8		24
10		a0	s9		25
11		a1	s10		26
12		a2	s11		27
13		a3	t3		28
14		a4	t4		29
15		a5	t5		30
			t6		31

Arguments and
functions support

RISC-V 32 register file

0		zero	a6		16
1		ra	a7		17
2		sp	s2		18
3		gp	s3		19
4		tp	s4		20
5		t0	s5		21
6		t1	s6		22
7		t2	s7		23
8		s0 / fp	s8		24
9		s1	s9		25
10		a0	s10		26
11		a1	s11		27
12		a2	t3		28
13		a3	t4		29
14		a4	t5		30
15		a5	t6		31

Pointers to memory “areas” (segments)

Register File (integers)

ABI Name	Number	Uso
zero	x0	Constant 0
ra	x1	Return address (routines)
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0...t2	x5...x7	Temporary (NON-kept between calls)
s0/fp	x8	Temporary (kept between calls) / Frame stack pointer
s1	x9	Temporary (kept between calls)
a0...a1	x10...x11	Input argument for routines/returned values
a2...a7	x12...x17	Input argument for routines
s2...s11	x18...x27	Temporary (kept between calls)
t3...t6	x28...x31	Temporary (<u>NON</u> -kept between calls)

- ▶ There are 32 registers
 - ▶ Size of 4 bytes (one word)
 - ▶ Dual name:
 - ▶ Logical: ABI name (*Application Binary Interface*)
 - ▶ Numeric: starting with **x** at the beginning
- ▶ Use agreement
 - ▶ Reserved
 - ▶ Arguments
 - ▶ Results
 - ▶ Temporary
 - ▶ Pointers

Register File (floating-point)

ABI Name	Name	Usage
ft0...ft7	f0 ... f7	Temporals (like t...)
fs0...fs1	f8 ... f9	There are saved (like s...)
fa0...fa1	f10 ... f11	Arguments/return (like a...)
fa2...fa7	f12 ... f17	Arguments (like a...)
fs2...fs11	f18 ... f27	There are saved (like s...)
ft8...ft11	f28 ... f31	Temporals (like t...)

- ▶ There are 32 registers
 - ▶ The ft0 register does not have its value set to 0.
- ▶ In the single precision extension, the registers are 32 bits (4 bytes).
- ▶ In the double precision extension, the registers are 64 bits (8 bytes) and can store:
 - ▶ Single precision values in the lower 32 bits of the register
 - ▶ Double precision values in the 64 bits of the register

CREATOR

Register
file

The screenshot displays the CREATOR simulator interface. On the left, a table lists instructions with their addresses, user instructions, and loaded instructions. The instruction at address 0xC is highlighted. On the right, the 'Registers' panel shows the state of various registers. An arrow points from the 'Register file' text to the 'Integer registers' section of the register panel.

Address	User Instruction	Loaded Instructions
0x0	li t0, 10	addi t0, x0, 10
0x4	li t1, 13	addi t1, x0, 13
0x8	li t2, 45	addi t2, x0, 45
0xC	li t3, 33	addi t3, x0, 33
0x10	add t4, t0, t1	add t4, t0, t1
0x14	sub t4, t2, t3	sub t4, t2, t3
0x18	mul t4, t3, t3	mul t4, t3, t3
0x1C	div t4, t2, t0	div t4, t2, t0
0x20	mv a0, t4	addi a0, t4, 0
0x24	li a7, 1	addi a7, x0, 1
0x28	ecall	ecall
0x2C	li a7, 10	addi a7, x0, 10
0x30	ecall	ecall

Registers

- > Control registers
- Integer registers
 - x0 | zero: 0
 - x1 | ra: -1
 - x2 | sp: 268435452
 - x3 | gp: 0
 - x4 | tp: 0
 - x5 | t0: 10
 - x6 | t1: 13
 - x7 | t2: 45
 - x8 | fp | s0: 0
 - x9 | s1: 0
 - x10 | a0: 0
 - x11 | a1: 0
 - x12 | a2: 0
 - x13 | a3: 0
 - x14 | a4: 0
 - x15 | a5: 0
 - x16 | a6: 0
 - x17 | a7: 0
 - x18 | s2: 0
 - x19 | s3: 0
 - x20 | s4: 0
 - x21 | s5: 0
 - x22 | s6: 0
 - x23 | s7: 0
 - x24 | s8: 0
 - x25 | s9: 0
 - x26 | s10: 0
 - x27 | s11: 0
 - x28 | t3: 33
 - x29 | t4: 0
 - x30 | t5: 0
 - x31 | t6: 0
- > Floating point registers

<https://creatorsim.github.io/>

Contents

1. Basic concepts on assembly programming
 1. Motivations and goals
 2. Introduction to RISC-V32
 - ▶ Registers
 - ▶ Data Transfer
 - ▶ Arithmetic and Logical Operations
 - ▶ Shifts
 - ▶ Branches or Jumps
2. RISC-V32 assembly language, memory model and data representation
3. Instruction formats and addressing modes
4. Floating point in assembly
5. Procedure calls and stack convention

Data transfer (integer registers)

- ▶ Copy data:
 - ▶ Between registers
 - ▶ Between registers and memory (later)

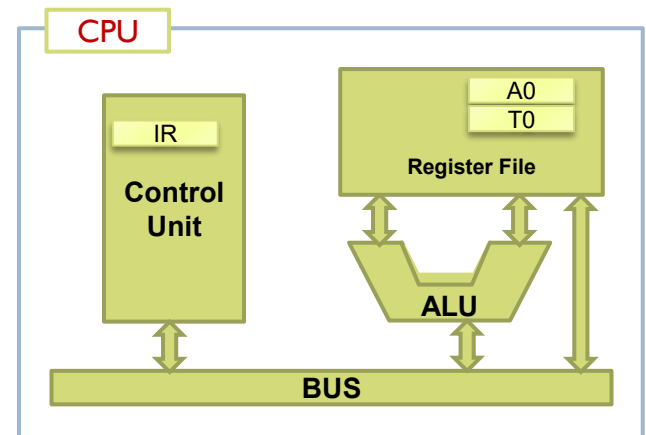
- ▶ Examples:

- ▶ Copy from register to register

`mv a0, t0`

- ▶ Immediate load

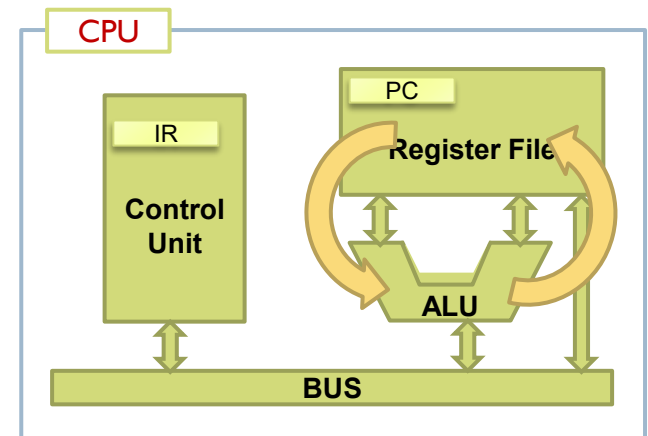
`li t0, 5`



```
mv    a0, t0    # a0 ← t0
li    t0, 5     # t0 ← 000....00101
```

Arithmetic instructions (integer registers)

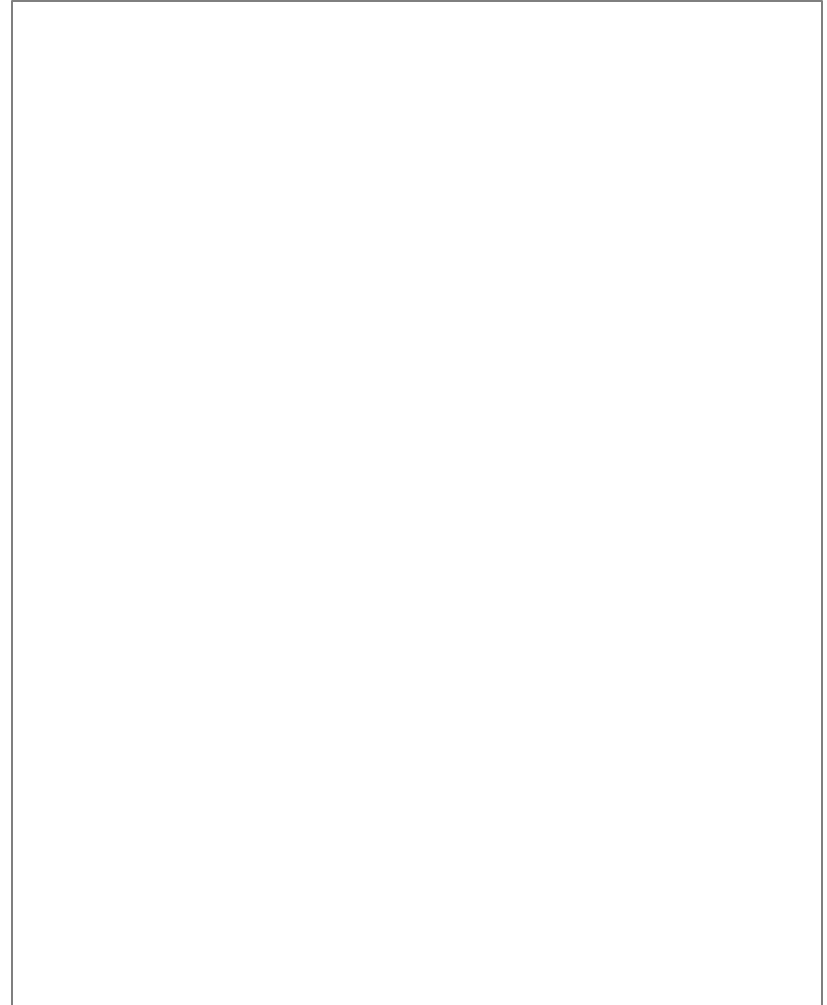
- ▶ Performs the **arithmetic integer operations in two complement**.
- ▶ Examples (ALU):
 - ▶ Addition
 - `add t0, t1, t2` $\# t0 \leftarrow t1 + t2$
 - `addi t0, t1, 5` $\# t0 \leftarrow t1 + 5$
 - ▶ Subtraction
 - `sub t0, t1, t2` $\# t0 \leftarrow t1 - t2$
 - ▶ Multiplication
 - `mul t0, t1, t2` $\# t0 \leftarrow t1 * t2$
 - ▶ Integer division (5 / 2=2)
 - `div t0, t1, t2` $\# t0 \leftarrow t1 / t2$
 - ▶ Division remainder (5 % 2=1)
 - `rem t0, t1, t2` $\# t0 \leftarrow t1 \% t2$



Example

```
int a = 5;  
int b = 7;  
int c = 8;  
int i;
```

```
i = a * (b + c)
```



Example

```
int a = 5;  
int b = 7;  
int c = 8;  
int i;
```

```
i = a * (b + c)
```

```
li  t1, 5 # t1:a  
li  t2, 7 # t2:b  
li  t3, 8 # t3:c
```

```
add  t4, t2, t3  
mul  t4, t4, t1
```

Exercise

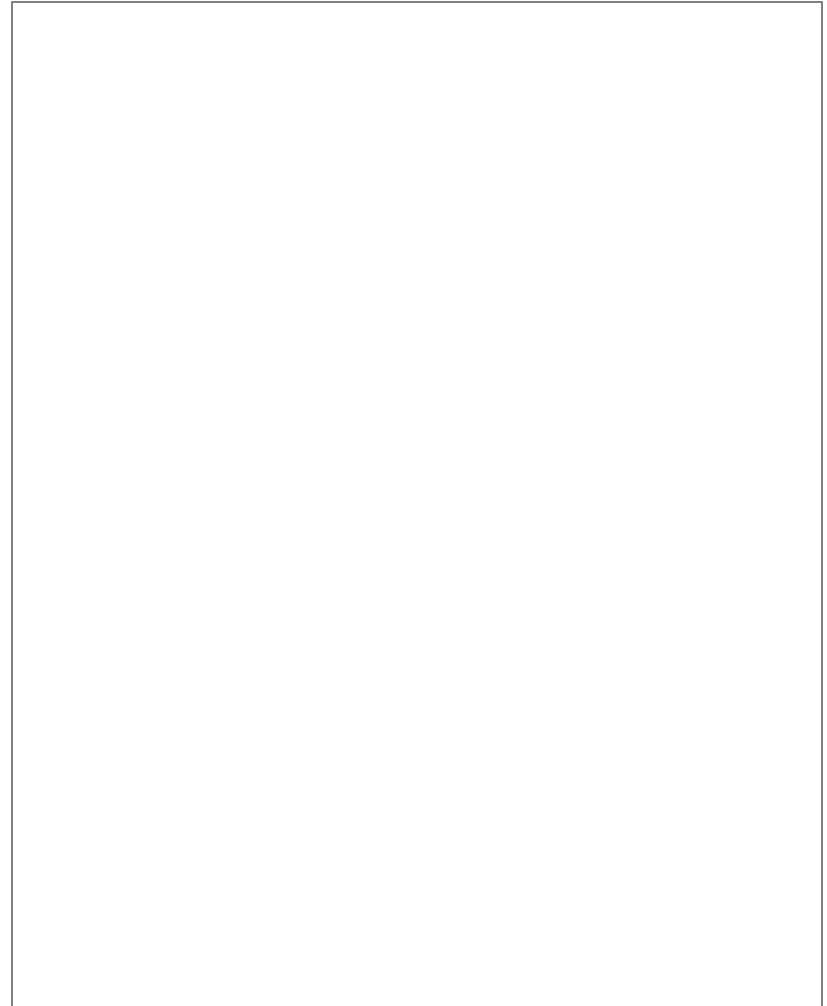
```
int a = 5;
```

```
int b = 7;
```

```
int c = 8;
```

```
int i;
```

```
i = -(a * (b - 10) + c)
```



Exercise (solution)

```
int a = 5;  
int b = 7;  
int c = 8;  
int i;
```

```
i = -(a * (b - 10) + c)
```

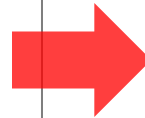
```
li  t1, 5  # t1:a  
li  t2, 7  # t2:b  
li  t3, 8  # t3:c
```

```
li    t0, 10  
sub   t4, t2, t0  
mul   t4, t4, t1  
add   t4, t4, t3  
li    t0, -1  
mul   t4, t4, t0
```

Exercise

```
li t1, 5  
li t2, 7  
li t3, 8
```

```
li    t0, 10  
sub   t4, t2, t0  
mul   t4, t4, t1  
add   t4, t4, t3  
li    t0, -1  
mul   t4, t4, t0
```

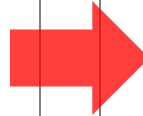


**Can be performed by using
less instrucciones?**

Exercise (solution)

```
li t1, 5  
li t2, 7  
li t3, 8
```

```
li    t0, 10  
sub   t4, t2, t0  
mul   t4, t4, t1  
add   t4, t4, t3  
li    t0, -1  
mul   t4, t4, t0
```



```
li t1, 5  
li t2, 7  
li t3, 8
```

```
addi  t4, t2, -10  
mul   t4, t4, t1  
add   t4, t4, t3  
add   t4, x0, t4
```

Logical instructions

- ▶ **Boolean** operations

- ▶ **Examples:**

- ▶ **AND**

- `and t0, t1, t2` ($t0 = t1 \& t2$)
 - `andi t0, t1, t2` ($t0 = t1 \& t2$)

- ▶ **OR**

- `or t0, t1, t2` ($t0 = t1 | t2$)
 - `ori t0, t1, 80` ($t0 = t1 | 80$)

- ▶ **NOT**

- `not t0, t1` ($t0 = ! t1$)
 - `xori t0 t1 -1`

- ▶ **XOR**

- `xor t0, t1, t2` ($t0 = t1 \wedge t2$)

	1100
AND	1010
	1000

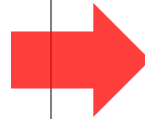
	1100
OR	1010
	1110

NOT	10
	01

	1100
XOR	1010
	0110

Exercise

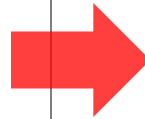
```
li t0, 5  
li t1, 0x007FFFFFFF  
  
and t2, t1, t0
```



What will be the value of t2?

Exercise (solution)

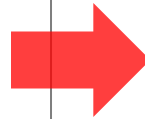
```
li t0, 5  
li t1, 0x007FFFFFFF  
  
and t2, t1, t0
```



```
00...0101    t0  
00...1111    t1  
00...0101  
and 00...0101    t2
```

Exercise

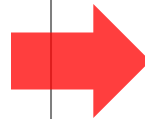
```
li t0, 5  
li t1, 0x007FFFFFFF  
  
and t2, t1, t0
```



What does an "and" with
0x007FFFFFFF allow to do?

Exercise (solution)

```
li t0, 5  
li t1, 0x007FFFFFFF  
  
and t2, t1, t0
```



What does an "and" with
0x007FFFFFFF allow to do?

Obtain the 23 least
significant bits

The constant used for bit
selection is called a
mask.

Contents

1. Basic concepts on assembly programming
 1. Motivations and goals
 2. Introduction to RISC-V32
 - ▶ Registers
 - ▶ Data Transfer
 - ▶ Arithmetic and Logical Operations
 - ▶ Shifts
 - ▶ Branches or Jumps
2. RISC-V32 assembly language, memory model and data representation
3. Instruction formats and addressing modes
4. Floating point in assembly
5. Procedure calls and stack convention

Shift instructions

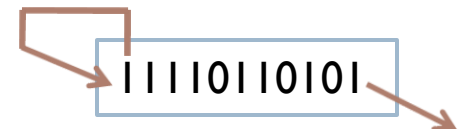
- ▶ Bits movement

- ▶ Examples:

- ▶ Shift **right logical**
`srl t0, t0, 4` ($t0 = t0 \ggg 4$ bits)

- ▶ Shift **left logical**
`sll t0, t0, 5` ($t0 = t0 \lll 5$ bits)

- ▶ Shift **right arithmetic**
`sra t0, t0, 2` ($t0 = t0 \ggg 2$ bits)

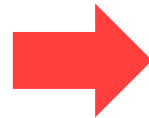


Exercise (solution)

```
li t0, 5
```

```
li t1, 6
```

```
srai t0, t1, 1
```



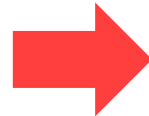
- What is the value of t0?

000 0110 t1

shift one bit to right (/2)

000 0011 t0

```
slli t0, t1, 1
```



- What is the value of t0?

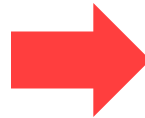
000 0110 t1

Shit one bit to left (x2)

000 1100 t0

Exercise

Make a program that detects the sign of a stored number t0 and leaves in t1 a 1 if it is negative and a 0 if it is positive.



Exercise (solution)

CREATOR

WepSIM

Make a program that detects the sign of a stored number t0 and leaves in t1 a 1 if it is negative and a 0 if it is positive.



```
li    t0, -3
srli  t1, t0, 31
```

Contents

1. Basic concepts on assembly programming
 1. Motivations and goals
 2. Introduction to RISC-V32
 - ▶ Registers
 - ▶ Data Transfer
 - ▶ Arithmetic and Logical Operations
 - ▶ Shifts
 - ▶ Branches or Jumps
2. RISC-V32 assembly language, memory model and data representation
3. Instruction formats and addressing modes
4. Floating point in assembly
5. Procedure calls and stack convention

Comparison instructions (integer registers)

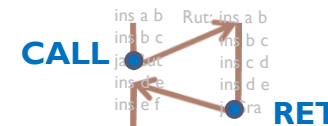
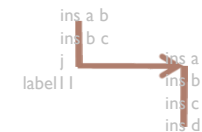
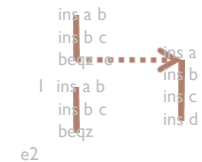
▶ <code>slt rd, rs1, rs2</code>	if (s(rs1) < s(rs2))	rd = 1; else rd = 0
▶ <code>sltu rd, rs1, rs2</code>	if (u(rs1) < u(rs2))	rd = 1; else rd = 0
▶ <code>slti rd, rs1, 5</code>	if (s(rs1) < s(5))	rd = 1; else rd = 0
▶ <code>sltiu rd, rs1, 5</code>	if (u(rs1) < u(5))	rd = 1; else rd = 0
▶ <code>seqz rd, rs1</code>	if (rs1 == 0)	rd = 1; else rd = 0
▶ <code>snez rd, rs1</code>	if (rs1 != 0)	rd = 1; else rd = 0
▶ <code>sgtz rd, rs1</code>	if (rs1 > 0)	rd = 1; else rd = 0
▶ <code>sltz rd, rs1</code>	if (rs1 < 0)	rd = 1; else rd = 0

as pseudoinstructions

lt: (<) less than
le: (<=) less equal than
gt: (>) greater than
ge: (>=) greater equal than
eqz: (==0) equal zero
nez: (!=0) not equal zero

Control flow

- ▶ Change the sequence of instructions to be executed
- ▶ Several types:
 - ▶ Conditional branches:
 - ▶ Branch if value match condition
 - ▶ E.g.: `bne t0, t1, label1`
 - ▶ Unconditional branches:
 - ▶ Always branch
 - ▶ E.g.: `j etiqueta2`
 - ▶ Function calls:
 - ▶ Branch with return
 - ▶ E.g.: `jal ra, subrutina1 ..... jr ra`



Branch instructions

- ▶ Conditional (only with integer registers):

- ▶ **beq** **t0** **t1** **label1** # jump to label1 if t0 == t1
- ▶ **bne** **t0** **t1** **label1** # jump to label1 if t0 != t1
- ▶ **blt** **t0** **t1** **label1** # jump to label1 if t0 < t1
- ▶ **bltu** **t0** **t1** **label1** # jump to label1 if t0 < t1 (unsigned)
- ▶ **bge** **t0** **t1** **label1** # jump to label1 if t0 >= t1
- ▶ **bgeu** **t0** **t1** **label1** # jump to label1 if t0 >= t1 (unsigned)
- ▶ **bgt** **t0** **t1** **label1** # jump to label1 if t0 > t1
- ▶ **ble** **t0** **t1** **label1** # jump to label1 if t0 <= t1

as pseudoinstructions

Branch instructions

jump label

► Conditional (only with integer registers):

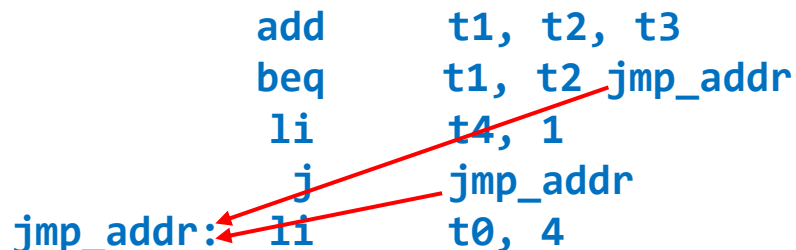
- `beq t0 t1 label1` # jump to label1 if `t0 == t1`
- `bne t0 t1 label1` # jump to label1 if `t0 != t1`
- `blt t0 t1 label1` # jump to label1 if `t0 < t1`
- `bltu t0 t1 label1` # jump to label1 if `t0 < t1` (unsigned)
- `bge t0 t1 label1` # jump to label1 if `t0 >= t1`
- `bgeu t0 t1 label1` # jump to label1 if `t0 >= t1` (unsigned)
- `bgt t0 t1 label1` # jump to label1 if `t0 > t1`
- `ble t0 t1 label1` # jump to label1 if `t0 <= t1`

► Unconditional:

- `j label1` # jump to label1. Equivalent to `beq x0, x0, label1`

`label1` refers to an instruction (represents a memory address where the instruction is located) which is skipped:

```
add    t1, t2, t3
beq    t1, t2 jmp_addr
li     t4, 1
j      jmp_addr
jmp_addr: li t0, 4
```



Control flow if

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int a=1;
int b=2;

main ()
{
    if (a < b) {
        a = b;
    }
    ...
}
```

Control flow if

CREATOR

WebSIM

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int a=1;
```

```
int b=2;
```

```
main ()
```

```
{  
  if (a < b) {
```

```
    a = b;
```

```
  }
```

```
  ...
```

```
}
```

```
li t1, 1
```

```
li t2, 2
```

```
if_1: blt t1, t2, then_1
```

```
j fin_1
```

```
then_1: mv t1, t2
```

```
fin_1: ...
```

Control flow if

CREATOR

WepSIM

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int a=1;
```

```
int b=2;
```

```
main ()
```

```
{  
  if (a < b) {
```

```
    a = b;
```

```
  }
```

```
  ...
```

```
}
```

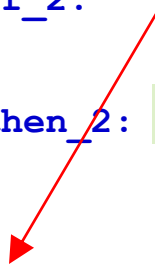
```
li t1, 1
```

```
li t2, 2
```

```
if_2: bge t1, t2, fin_2
```

```
then_2: mv t1, t2
```

```
fin_2: ...
```



Control flow if-else

CREATOR

WepSIM

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int a=1;
```

```
int b=2;
```

```
main ()
```

```
{
```

```
  if (a < b){
```

```
    // action 1
```

```
  } else {
```

```
    // action 2
```

```
  }
```

```
}
```

```
li t1, 1
```

```
li t2, 2
```

```
if_3: bge t1, t2, else_3
```

```
then_3: # action 1
```

```
j fi_3
```

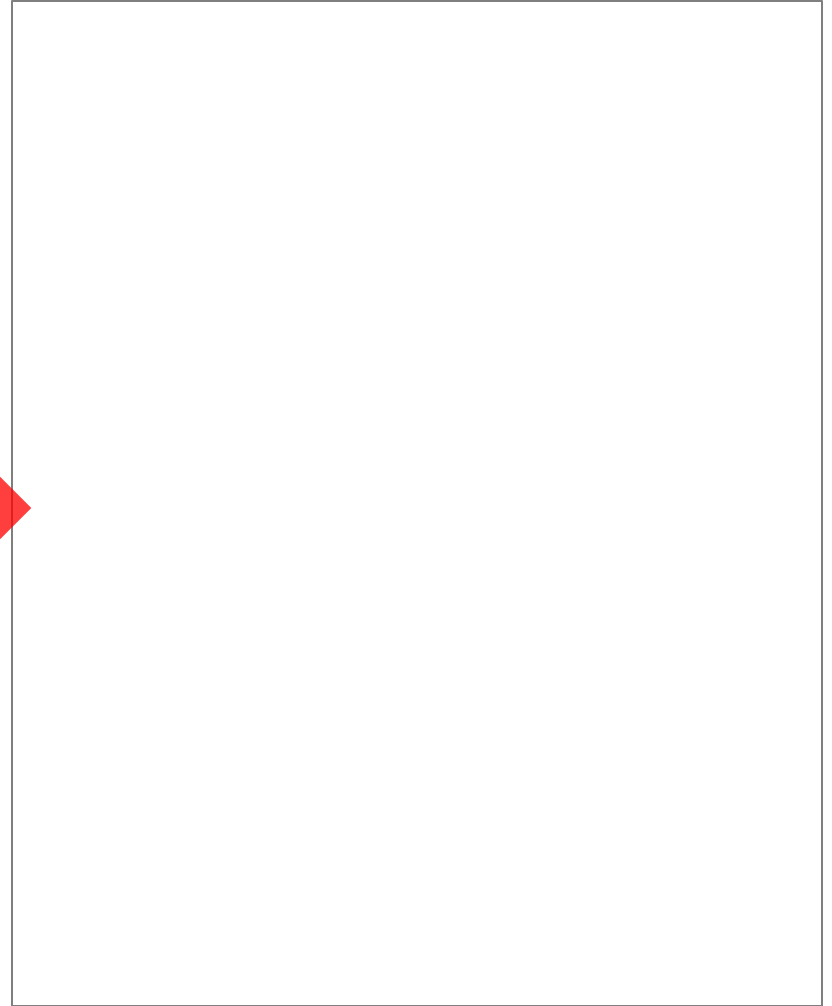
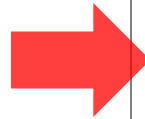
```
else_3: # action 2
```

```
fi_3: ...
```

Exercise

```
int b1 = 4;  
int b2 = 2;
```

```
if (b2 == 8) {  
    b1 = 1;  
}  
...
```



Exercise (solution)

```
int b1 = 4;  
int b2 = 2;
```

```
if (b2 == 8) {  
    b1 = 1;  
}  
...
```



```
li    t0, 4  
li    t1, 2  
li    t2, 8  
  
bne   t0, t2, fin1  
li    t1, 1  
fin1: ...
```

Control flow while

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int i;

main ()
{
    i=0;
    while (i < 10) {

        /* action */
        i = i + 1 ;
    }
}
```

Control flow while

CREATOR

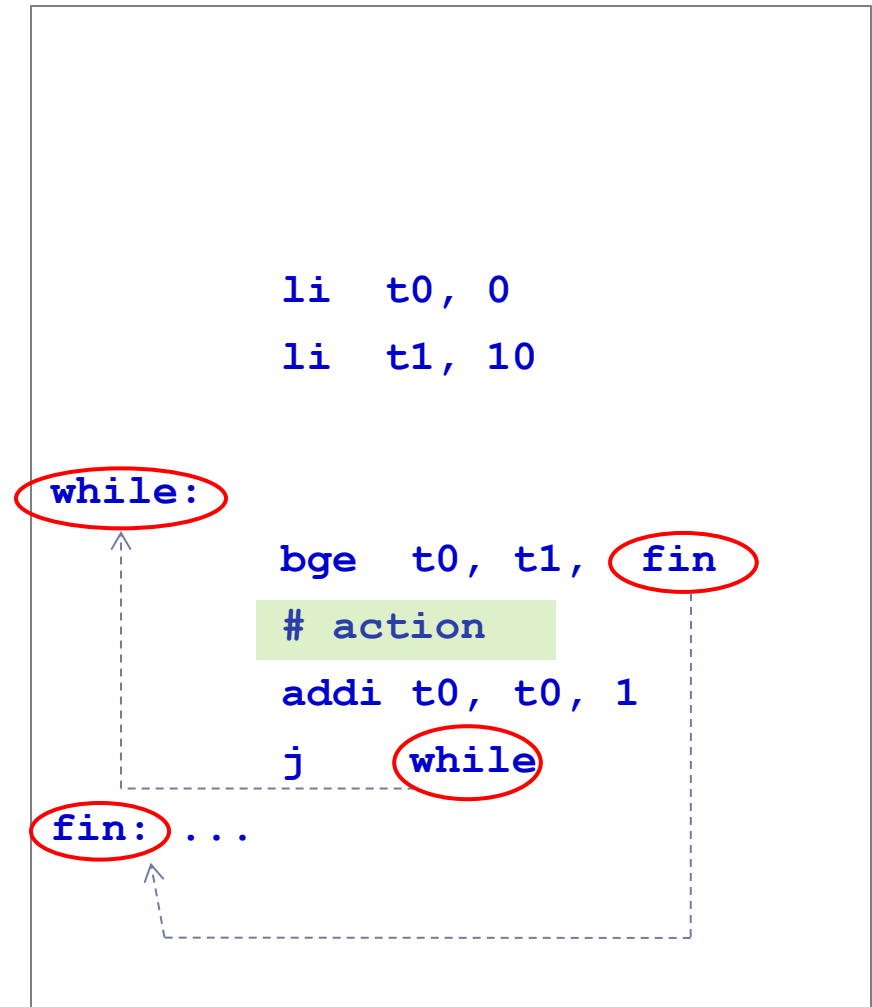
WepSIM

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int i;

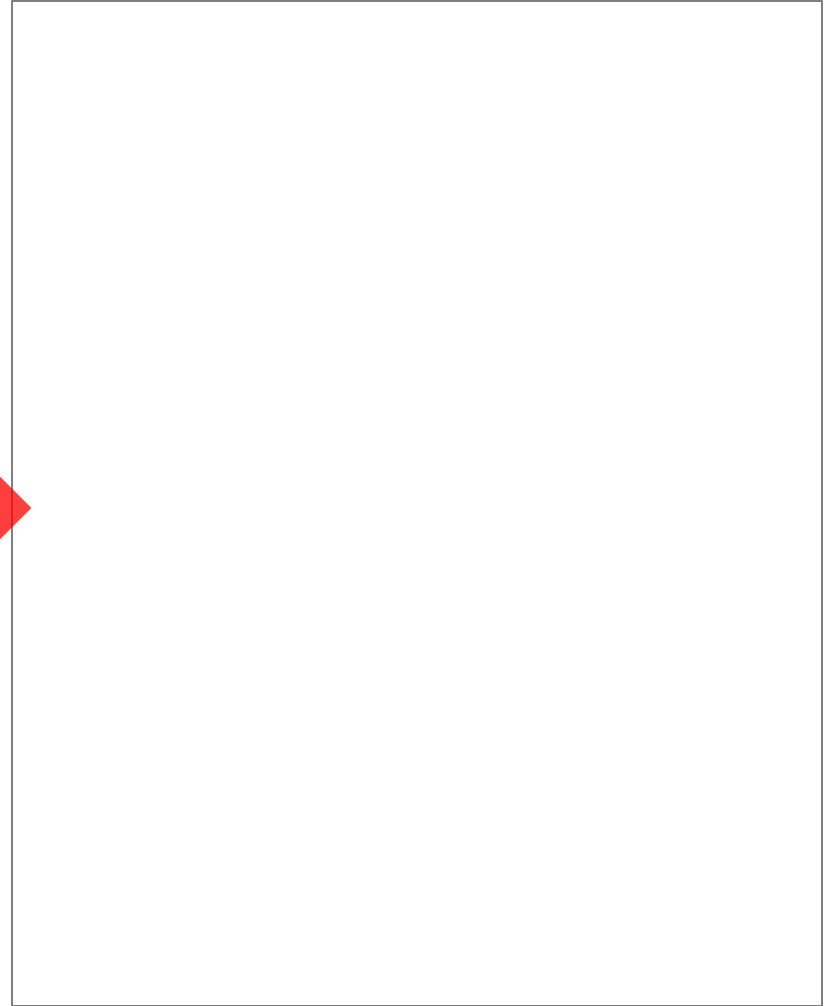
main ()
{
    i=0;
    while (i < 10) {

        /* action */
        i = i + 1 ;
    }
}
```



Exercise

Make a program that
calculates the sum of
the first ten numbers
and leaves this value in
register a0



Exercise (solution)

Make a program that calculates the sum of the first ten numbers and leaves this value in register a0

$1 + 2 + 3 + \dots + 10$



```
li  a0, 0
add a0, a0, 1
add a0, a0, 2
add a0, a0, 3
add a0, a0, 4
add a0, a0, 5
add a0, a0, 6
add a0, a0, 7
add a0, a0, 8
add a0, a0, 9
```

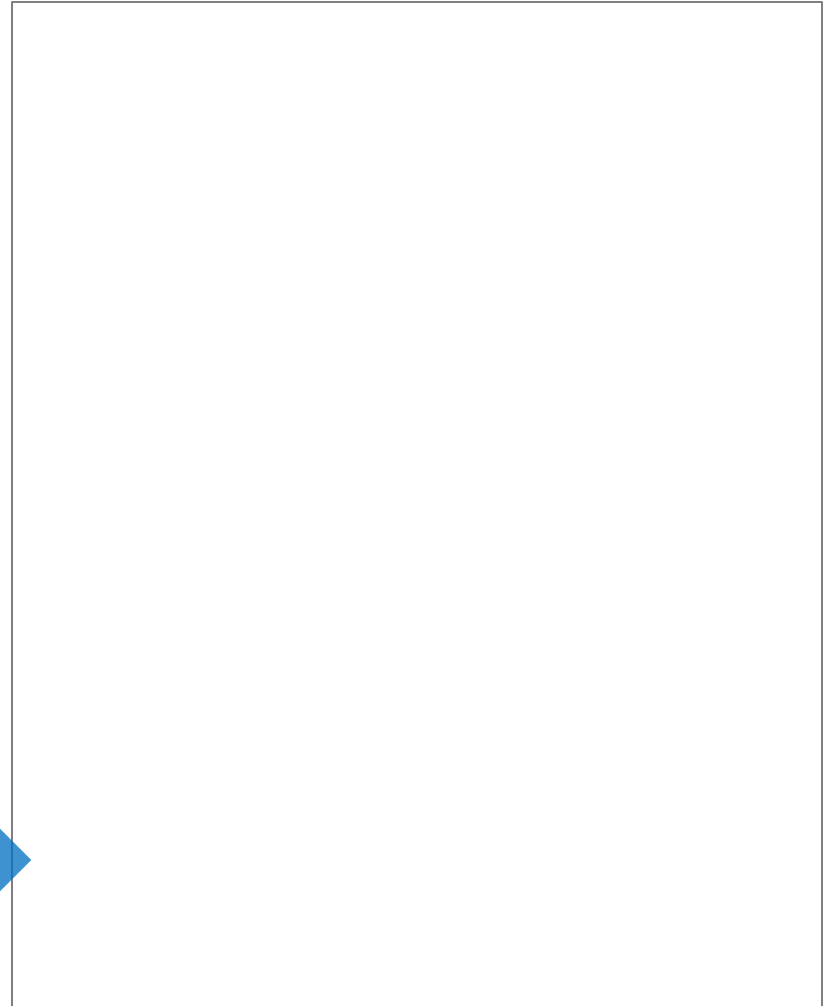
No valid
;-)

Exercise (solution)

Make a program that
calculates the sum of
the first ten numbers
and leaves this value in
register a0



```
int i, s;  
  
s=0;  
i=0;  
while (i <= 10)  
{  
    s = s + i ;  
    i = i + 1 ;  
}
```

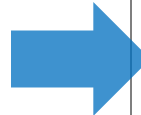


Exercise (solution)

Make a program that calculates the sum of the first ten numbers and leaves this value in register a0



```
int i, s;  
  
s=0;  
i=0;  
while (i <= 10)  
{  
    s = s + i ;  
    i = i + 1 ;  
}
```



```
li t0, 0  
li a0, 0  
li t2, 10  
  
while1:  
    bgt t0, t2, fin1  
    add a0, a0, t0  
    addi t0, t0, 1  
    j while1  
  
fin1:
```

Exercise

- ▶ Calculate the number of 1's of a register (t0).
Result in t3.

Exercise (solution)

- Calculate the number of 1's of a register (t0).
Result in t3.

```
i = 0;
n = 45;  # number
s = 0;
while (i < 32)
{
    b = first bit of n
    s = s + b;
    shift the contents of
      of n one bit to the
      right
    i = i + 1 ;
}
```

Exercise (solution)

- Calculate the number of 1's of a register (t0).
Result in t3.

```
i = 0;
n = 45;  # number
s = 0;
while (i < 32)
{
    b = first bit of n
    s = s + b;
    shift the contents of
      of n one bit to the
      right
    i = i + 1 ;
}
```

```
i = 0;
n = 45;  # number
s = 0;
while (i < 32)
{
    b = n & 1;
    s = s + b;
    n = n >> 1;
    i = i + 1 ;
}
```

Exercise (solution)

- Calculate the number of 1's of a register (t0).
Result in t3.

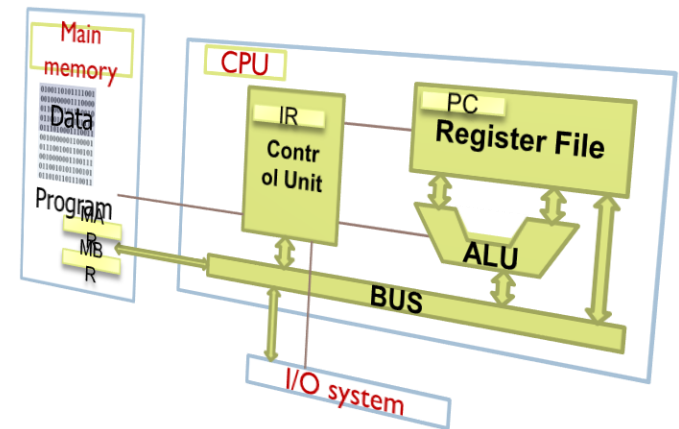
```
i = 0;
n = 45;  # number
s = 0;
while (i < 32)
{
    b = n & 1;
    s = s + b;
    n = n >> 1;
    i = i + 1 ;
}
```

```
li    t0, 0    #i
li    t1, 45   #n
li    t2, 32
li    t3, 0    #s
while: bge     t0, t2, fin
      andi    t4, t1, 1
      add     t3, t3, t4
      srli    t1, t1, 1
      addi    t0, t0, 1
      j       while
fin:   ...
```

Types of instructions

summary

- ▶ Data transfer
- ▶ Arithmetic
- ▶ Logical
- ▶ Shifting and rotation
- ▶ Comparison
- ▶ Control flow (branches, ...)
- ▶ Conversion
- ▶ Input/output
- ▶ System calls



Typical faults

1) Poorly designed program

- ▶ Does not do what is requested
- ▶ Incorrectly does what is requested

2) Programming directly in assembler

- ▶ Do not code in pseudo-code the algorithm to be implemented

3) Write unreadable code

- ▶ Do not tabulate the code
- ▶ Do not comment the assembly code or make reference to the algorithm initially proposed.

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L3: Fundamentals of assembler programming (I) **Computer Structure**

Bachelor in Computer Science and Engineering
Bachelor in Applied Mathematics and Computing
Dual Bachelor in Computer Science and Engineering and Business Administration

